
5.1 Introduction

In the previous chapter it was seen that lists can be created using arrays. However, there are some apparent disadvantages when inserting or deleting an item when the contents of the list must be moved to make way for the new item or filling in an empty space respectively. There is also a problem when an array becomes full where either an item cannot be inserted or the array must be increased in size. In all of these cases, it takes time to accomplish the task at hand.

A solution is to use what is known as a linked list. A *linked list* can be implemented using an array or using references and objects. Although a linked list using an array can be implemented in any language, it can be somewhat complicated and does not eliminate all of the problems of non-linked lists mentioned above. For languages that support references, such as Java, it is easier to understand linked list concepts and they do not require the writing of as many methods.

Although linked lists are faster for insertion or deletion, and do not suffer the problem of trying to decide how large of an array should be declared, they do have their disadvantages. Although in many cases linked lists are faster than non-linked lists implemented in arrays, they use more memory for the increase in speed. In other words, the major disadvantage is that they sacrifice memory for speed. Although generally faster, another disadvantage is that they cannot be searched as quickly even when an ordered list is used, since a binary search cannot be used.

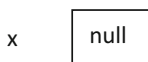
In spite of these limitations, linked lists are very useful and are frequently employed in computers. Some of these uses are storing data on disks or in memory, and are utilized extensively in operating systems. Before actually implementing a linked list in the next chapter, there are a number of concepts that must be discussed.

5.2 References

Before looking at a linked list, it is important to review the concept of a reference. Recall from a previous course or text on Java, when creating an array, a variable is declared that can refer to an array.

```
int[] x;
```

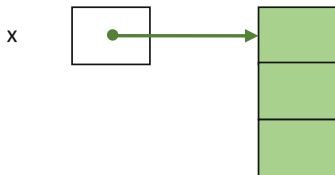
Initially `x` contains a `null` reference as shown below:



Then when the following statement is executed,

```
x = new int[3];
```

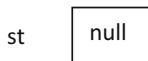
a three element array of integers is created and a reference (represented by the arrow) is assigned to the variable `x`.



Similarly, when a variable of type `String` is declared as follows:

```
String st;
```

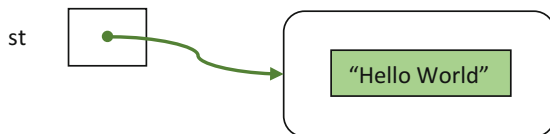
The variable `st` initially contains a `null` reference:



Then when the following instruction is executed:

```
st = new String("Hello World");
```

a `String` object is created by the `new` instruction and a reference to the string is placed into the variable `st` as follows:



As should be familiar to the readers of the previous text, *Guide to Java* [2], the outside rounded rectangle is a contour which represents an object of type `String` and within the contour is the text data.

5.3 Objects

As with strings and as a review, the same thing occurs when one creates an object. The class below is called `Node` because as will be seen in the next section, each object in a linked list is typically called a *node* and it is helpful to have the class have the same name. Likewise, the data member is called `data`, so again it helps to use that name as well. The class also contains two constructors: one which accepts a parameter to initialize `data` and the other that invokes the previous constructor to initialize `data` to 0. The class also contains the usual `getData` and `setData` methods.

```
public class Node {  
    private int data;  
  
    public Node() {  
        this(0);  
    }  
}
```

```

public Node(int data) {
    this.data = data;
}

public int getData() {
    return data;
}

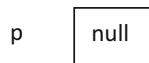
public void setData(int data) {
    this.data = data;
}
}

```

As in the previous section, the following declaration of the variable `p` is of type `Node`:

```
Node p;
```

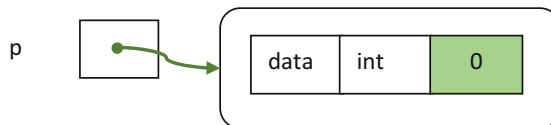
Again the variable `p` would contain a `null` reference inside the main method. Although a complete contour for the main method could be drawn, only a simple labeled box will be drawn for the variable `p` in the main method as done with both `x` for an array and `st` for a string previously, and as shown below:



Then when the following instruction is executed,

```
p = new Node();
```

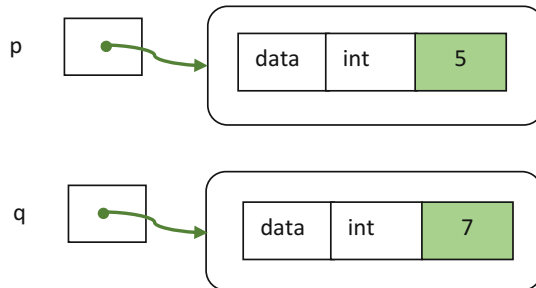
a new object of type `Node` is created and a reference to the object is assigned to the variable `p`:



The above contour represents the object containing the data member `data`. As before the arrow represents the reference to the instance of the `Node` class. To create a second instance of the `Node` class, a second variable can be declared and another object assigned to it as given in the following code segment:

```
Node p, q;  
p = new Node(5);  
q = new Node(7);
```

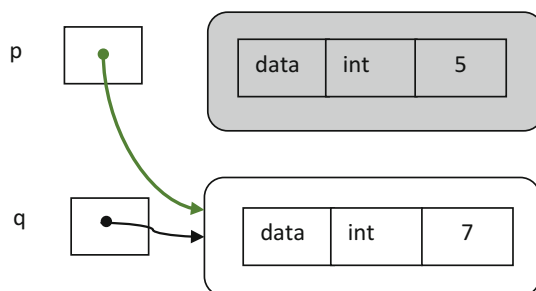
which would create the following contour diagrams:



However, what would happen if the following instruction was executed?

```
p = q;
```

Would the variable `data` inside `p` contain the number 7? The answer is no, because the contents of `data` in the object referenced by `q` are not copied into `data` in the object `p`, but rather the contents of `q`, which is a reference to the object containing the 7, would be copied into `p`. In other words, it is not the contents of the object that are copied, but rather the contents of the variable `q` that are copied. Since `q` contains a reference to an object, it is a copy of the reference to that object that is copied, not the object itself. The result is that since `q` refers to the object with the 7, `p` would also refer to the object with the 7 as indicated by the green arrow in the contour diagram below:



What would happen to the object containing the 5? Is it still accessible? The answer is no and the object will be returned to the heap by the garbage collector as indicated by the contour now being gray in color. The reason why many beginning

programmers sometimes think that the 7 was copied is that when they execute the following `println` statement, a 7 is output causing them to mistakenly think that 7 was copied.

```
System.out.println("p = " + p.get());
```

However, to prove that it was not copied, assume that the following statement is executed:

```
q.setData(9);
```

What would be output if the preceding `println` statement was executed again? Would the number 7 still be output? It might be thought that since the data in `q` was changed to 9 that the `p` would not change. However, the answer would be no, because `p` and `q` are still referring to the same object and the output would be 9.

This is why it is important to have a good mental model of objects and references by using drawings such as contour diagrams, because without them one will have difficulties understanding how references work and subsequently understanding linked lists. For more information on or for a review of contours, see Chaps. 3 and 6 in *Guide to Java* [2]. Although exploring references with simple nodes like these are interesting in and of themselves, they are just a step in learning how to create a linked list.

5.4 The Node Class

In order to link nodes together, the `Node` class will need to be modified to contain another data member to hold a reference such as the following:

```
Node next;
```

It is called `next` because it will be used to reference the next node in the linked list. But what might be confusing is that it is declared to be of type `Node`. Is it legal to have a data member be of the same type as the class that it is declared in? The answer is yes, because that is how one will be able to link the objects together as will be seen in the next section.

In addition to the above data member, there will need to be the appropriate modification to the initializing constructor to set the `next` data member to `null`.

```
public Node(int data) {  
    this.data = data;  
    next = null;  
}
```

In addition there will need to be two new methods. One to get the contents of `next` as shown below:

```
public Node getNext() {  
    return next;  
}
```

and there should also be the ability to set the `next` data member to refer to another node as follows:

```
public void setNext(Node next) {  
    this.next = next;  
}
```

Lastly, adding two constructors along with all of the above to the previous `Node` class results in the new `Node` class in Fig. 5.1.

Given the new `Node` class it is now possible to create links between instances of the `Node` class in the following section.

Fig. 5.1 Node class

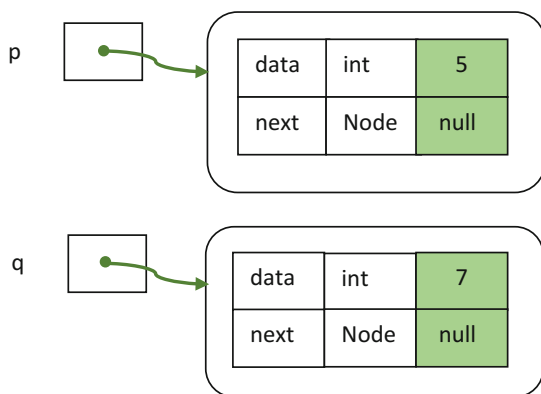
```
public class Node {  
    private int data;  
    private Node next;  
  
    public Node() {  
        this(0);  
    }  
  
    public Node(int data) {  
        this.data = data;  
        next = null;  
    }  
  
    public int getData() {  
        return data;  
    }  
  
    public void setData(int data) {  
        this.data = data;  
    }  
  
    public Node getNext() {  
        return next;  
    }  
  
    public void setNext(Node next) {  
        this.next = next;  
    }  
}
```

5.5 Creating Links

Starting again with the code segment from Sect. 5.3,

```
Node p, q;  
p = new Node(5);  
q = new Node(7);
```

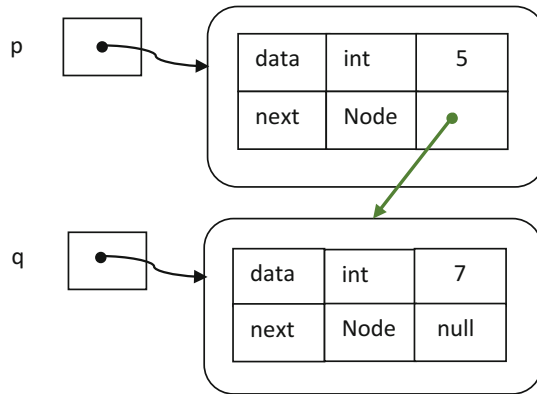
the following would be created using the new `Node` class. However, this time note the inclusion of the data member `next` in each object.



Further, notice that the `next` data member is of type `Node` which means it can refer to objects of type `Node`. The result is that the object referred to by `p` can now refer to the object referred to by `q`. How can this be accomplished? The intent is to have `next` in the object referred to by `p` refer to the object that is referred to by `q`. So, the `setNext` method must be used as follows:

```
p.setNext(q);
```

This causes the `next` data member in the object referred to by `p` to be set the same as `q` as follows:



In other words, since *q* references the node with the 7 in it, so too does the *next* field in the object referred to by *p*. It sometimes confuses programmers learning about links as to what is in fact being copied. For example, if a person in a room is pointing at the door and you are going to copy what they are doing, you would be pointing to the door also.

Instead of using arrows, if *q* is hypothetically referring to memory location 200, the *next* field would refer to memory location 200 also. In the following diagram, the top contour is at memory location 100 and the bottom one is at memory location 200. The variable *p* is referencing 100 as indicated by the 100 in parentheses and *q* is referencing 200 as indicated by the 200 in parentheses. When *p.setNext(q)* is executed, then the (200) in *q* is copied into *next* in the contour referenced by *p* as shown below:

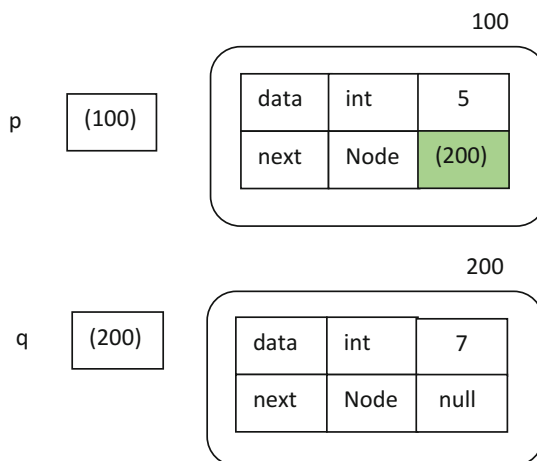
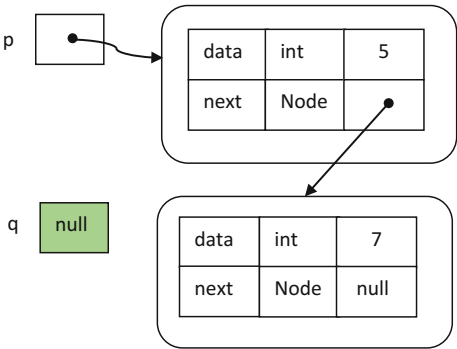
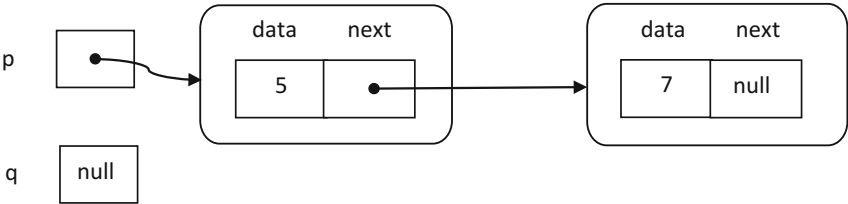


Fig. 5.2 List using references and objects

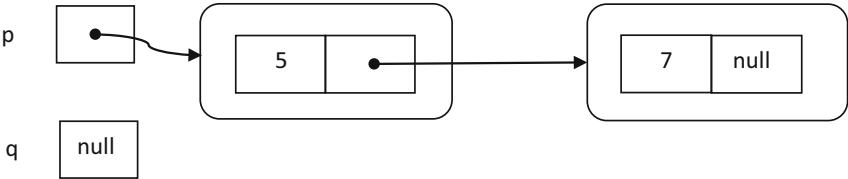


Either addresses or arrows can be used, but one will find the arrows are much easier to work with in subsequent examples. Returning back to the arrows instead of addresses, the result is that it is the base of the pointer that is copied into the new memory location. Since the reference in `q` is no longer needed, it can be set to `null` and a small linked list has been created as shown in Fig. 5.2.

Although contours are extremely helpful with understanding objects, when learning about linked lists and trying to understand more complicated combinations of nodes, the use of contours can become somewhat cumbersome. To that end, the following will be a series of drawings that remove some of these details a step at a time. First, note that the types could be removed, the boxes reorganized, and the names of the cells placed above the cells as shown below:



So that each memory cell does not need to be labeled, it could be assumed that the left cell will always be the `data` memory location and the right cell will always be the `next` memory location as shown below:



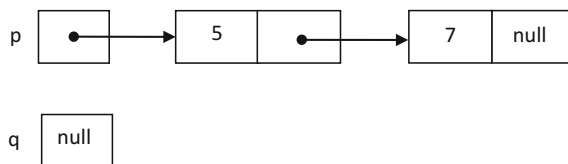
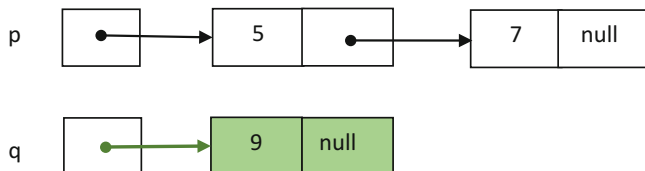


Fig. 5.3 A Simple linked list

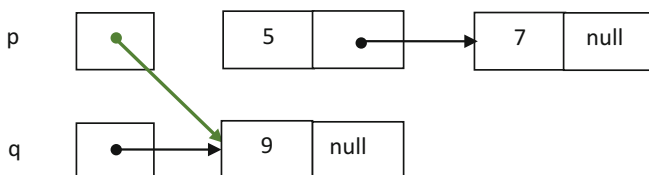
Of course, if needed the labels could always be reintroduced. Notice that the references in this drawing and in each of the past three drawings are not pointing at the data members but rather at the contours representing the nodes. This is an important distinction, because references do not point at the individual data members, but rather at the entire node. However, just as before when the contours for the main method were not drawn, it can be helpful not to draw the contours for the nodes to help simplify the linked list drawings as in Fig. 5.3.

The above drawing at first might seem to contradict what was just said in the last paragraph. It appears that the references are no longer pointing at a node, but rather an individual data member within the node. Is the above drawing correct? As before, sometimes the use of contours can complicate an already complex drawing. As a result, it is sometimes helpful to remove them. However, the problem is that the references can no longer point to the edge of the contour representing the node, but rather only to one of the cells within the node. Does this mean that it is now pointing at an individual data member? The answer is no. Instead what one has to do is imagine that there is an invisible contour surrounding the entire node and pointing at one data member is no different than pointing to any other data member within the node. Although typically the arrows will point at the data cell, one must remember that when a reference is pointing at a particular cell it is not just pointing at a cell but rather the entire node. In confusing instances and if needed, all the contours can be included, just as the labels and types can also be included as shown previously in Fig. 5.2. Again, when the pictures get more complicated, it helps remove some of the other aspects of the drawing to help see the "big picture" as in Fig. 5.3.

To continue, what if one wanted to insert a node containing a 9 at the beginning of the above list? Although it might appear that the 9 should belong at the end of the list, assume this is an unordered linked list so where it is placed is irrelevant and the beginning of the list is probably the easiest place to insert it. To accomplish this task, first a new node containing the number would need to be created using `q = new Node(9)`; which would look as follows:

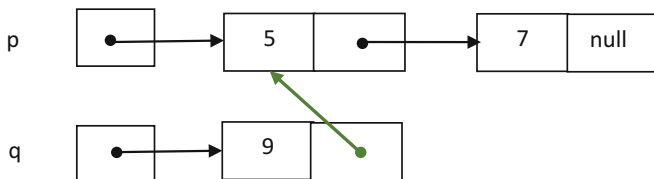


Then, in examining the picture, what needs to be done to insert the node containing the 9 at the beginning of the list? At first, one might think that the reference in p would need to reference the node containing the 9, using $p = q$; as shown in the following diagram. But what is wrong with this scenario?



Notice that although p is now referencing the node with the 9, there is no longer a reference to the node containing the 5 and the rest of the list. What would happen to the rest of the list? If one responded that it is now no longer accessible, one would be correct. Technically, since nothing is referencing the list, the garbage collector would return those nodes to the heap to be reused. Although p eventually needs to refer to the same node as q , this illustrates that the order in which operations are performed is just as important as which operations need to be performed.

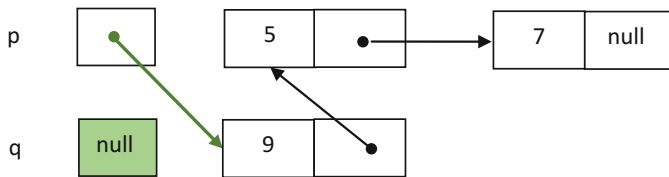
Instead, a general rule of thumb is to establish new links before breaking old links. In the situation above, the node containing the 9 first needs to be linked to the node containing the 5 before p refers to the node containing the 9. The way this is accomplished is to use $q.setNext(p)$; which would look as follows:



Now that the link connecting the node with the 9 to the node with the 5 has been established, it is now safe to have p reference the same node as q and have q set back to `null` in the following code segment:

```
p = q;  
q = null;
```

And as shown below:



Although not necessary, the drawing can be redrawn so that the list is on one horizontal line:

Other possible manipulations of this list, such as removing the node containing the number 5 or adding a node containing the number 3 between the 9 and 5, are left as exercises at the end of this chapter.

5.6 Output

Given the linked list in Fig. 5.4, it would be helpful to be able to output the contents of the list. Since there are only three elements in the list, it would be simple to just output the first item in the list as follows:

```
System.out.println(p.getData());
```

However, in the most recent drawing of the list, `q` had been set to `null`, so how could the contents of the second element be output? One method would be to set `q` to reference the second node in the list. But how could this be done? The only route to the second node is to go through `p`. The next cell in the node referenced by `p` is referencing the node with the 5, therefore `p` would have a copy of that reference as well. So to have `q` reference the second node the following would work:

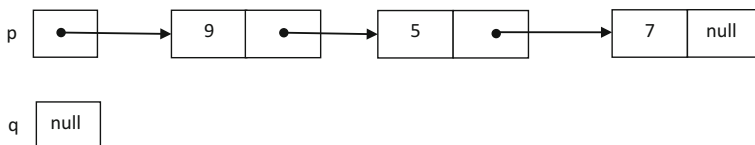


Fig. 5.4 A Singly linked list

```
q = p.getNext();
```

And the following `println` would output the contents of data 5.

```
System.out.println(q.getData());
```

However, this could be shortened where the variable `q` would not need to be used by combining the two `get` methods in the `println` statement as follows:

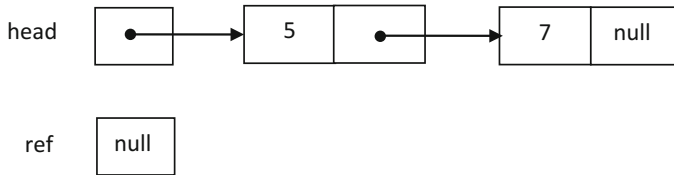
```
System.out.println(p.getNext().getData());
```

In other words by following the reference out of `p`, one can get the contents of `next`, which is a reference to the second node, and `getData` would get the number 5 to be output by the `println`.

Although this way is a little cleaner because it does not need to use another variable, such as `q`, and it only takes up one line, what about the third node in the list? The `println` statement for each node would need to have another `getNext` and each successive `println` statement could become rather unwieldy. Further, if the number of nodes were unknown, it would be difficult to create a program to output `n` nodes.

As one might suspect, the solution to such a problem involving `n` things to be processed is a loop. Although a `for` loop could be used, it is a little easier in this case to initially use a `while` loop. However, instead of iterating `n` times requiring the use of a count as done previously with the list using an array, the linked list above has a built in terminator with the last node containing a `null` in the `next` cell. Although a count could be kept if needed, it is not necessary in this instance. Further, the loop will not need a variable such as `i` as a loop control variable to count the number of items (unless a count of the number of nodes is needed), nor will it be needed as an index since there is obviously no array.

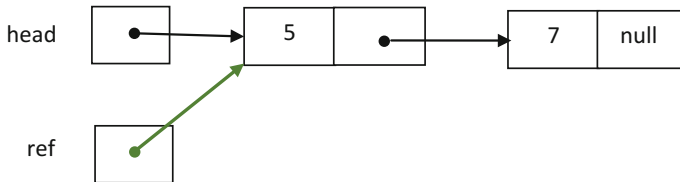
Instead a variable capable of referencing nodes is needed. Renaming some of the variables from the previous figure, often the variable referencing the first node in a linked list is called `first` or `head` indicating the first node or the head of the linked list. Then another variable is needed for moving down or in other words traversing the length of the list. In other languages which use pointers, this variable has been called `ptr` for pointer, but since Java uses references this text will use the abbreviation `ref` for reference or alternatively `curr` as an abbreviation for current. Replacing `p` and `q` from the two-element simple linked list in Fig. 5.3 with `head` and `ref`, the new drawing is as follows:



Recall from a first course or text in programming the three parts of most loops are initialize, test, and change. The first thing that needs to be done is to initialize the loop control variable (LCV) which in this case is `ref`. It will need to be initialized to the first node in the list, which is referenced by `head`.

```
ref = head;
```

Thus the drawing would be as follows:



Then for each time through the loop there needs to be a test to terminate the loop. In the case of an empty list, such as at the beginning of this chapter, the value for `head` would be `null` and when the above instruction is executed, `ref` would also contain a `null`, and the loop would terminate. So, the loop should continue iterating while `ref` is not equal to `null` as follows:

```
while(ref != null)
```

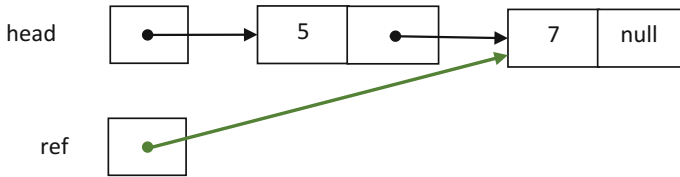
In the previous drawing, `ref` is not equal to `null` so the contents of the node could be output as shown below:

```
System.out.println(ref.getData());
```

Now the only thing remaining is the change portion of the loop which requires the value of `ref` to move to the next node. In other words the value of `ref` needs to take on the value of `next` in the node that `ref` is referring to. Putting this into code results in the following:

```
ref = ref.getNext();
```

Again, the value of `next` in the node that `ref` is currently referencing is copied into `ref`, thus causing the old value of `ref` to be updated with a reference to the second node in the list as follows:



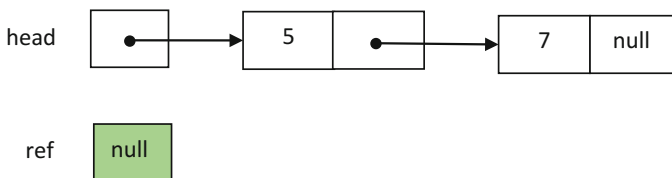
Putting all of the code together results in the following segment:

```

ref = head;
while(ref != null) {
    System.out.println(ref.getData());
    ref = ref.getNext();
}

```

After the first iteration of the loop, control returns back to the `while` statement, where `ref` is not equal to `null`, so the body of the loop executes again. The 7 from the data cell of the node is output by the `println` statement and the `next` instruction is executed. After the second iteration, the value in `next` in the node referenced by `ref` contains a `null` and this is copied into `ref` as shown below:



This time when the flow of control returns back to the `while` statement, it is `false` since `ref` is now equal to `null`, the loop ceases to iterate, and the contents of the list have been output.

5.7 Test Program

Instead of creating each individual node as they are needed, as done previously, wouldn't it be nice to be able to use the power of an iteration structure to create a list similar to the way an iteration structure was used to output a list? The program could utilize a sentinel control loop to prompt for and input numbers that could then be placed in a linked list. Although an ordered linked list will be discussed in the

next chapter, this does not preclude creating an unordered linked list here. Further, a `LinkedList` class will also be introduced in the next chapter, so for now just a main program will be used to create this test linked list.

Since only integers are being used here, it might be helpful to assume that only non-negative numbers will be entered into the list, so if a negative number is entered, the loop will stop and the program can continue on to output the list. While non-negative numbers are entered, a new node is created and a reference to the node is placed in a variable called `newp`. Then the `next` field of the new node is made to reference the beginning of the list and `head` is made to reference the new first node in the list, similar to the process in Sect. 5.5. Once a negative number is entered, the loop stops iterating, the column heading is output, and the list is output as done in Sect. 5.6. The following code segment implements this algorithm:

```
int number;
Node head, newp, ref;

head = null;
System.out.println();
System.out.print("Enter a negative integer to stop: ");
number = scanner.nextInt();
while(number >= 0) {
    newp = new Node(number);
    newp.setNext(head);
    head = newp;
    System.out.print("Enter a negative integer to stop: ");
    number = scanner.nextInt();
}
System.out.println();
System.out.println(" Unordered");
System.out.println("Linked List");
System.out.println();
ref = head;
while(ref != null) {
    System.out.println("      " + ref.getData());
    ref = ref.getNext();
}
```

Implementing and executing the above program with some sample data yields the following output:

```
Enter a negative integer to stop: 9
Enter a negative integer to stop: 5
Enter a negative integer to stop: 7
Enter a negative integer to stop: -1
```

```
Unordered
Linked List

7
5
9
```

In examining the above output, is there a noticeable pattern? Yes, it should be noticed that the data that was input is output in reverse order. Further, is there a particular data structure that behaves in this manner? Yes, as one recalls from Chap. 2, the stack can be useful when one needs to output data in reverse order, where here a linked list is used instead of an array. Although the above is not a complete implementation of a stack, it illustrates that linked lists can be used to implement a variety of data structures, and the stack will be revisited in Chap. 7.

5.8 Output Using Recursion

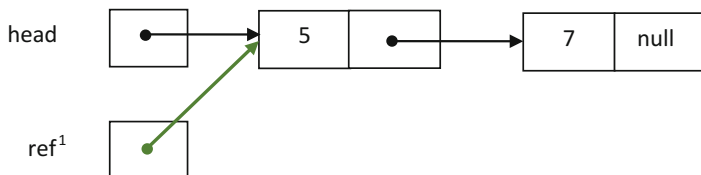
The preceding section illustrated how the contents of the list can be output using iteration, but as illustrated in *Guide to Java* [2] and other texts, many problems that can be solved using iteration can be solved using recursion. Instead of just calculating a number such as x^n or $n!$, recursion can be extremely useful in processing or outputting data from linked structures. Using the same list from the preceding section, how could recursion be used to traverse and output the contents of each node? As with trying to solve problems utilizing just numbers, such as $n!$, it helps to first recognize a pattern. Using the two-element list in Fig. 5.3, notice that if the first node in the list is processed or output, what is remaining? The answer is a list, but this time with only one node in it. And if that node is processed or output, what is remaining? Still another list, but this time it is an empty list or `null`. This last case is known as the base case or terminal case because should the reference be `null` then the recursion ought to stop. Instead of using a loop to test for the terminal case, an `if` statement can be used, where if the reference is not `null`, then the contents of the node would be output and the reference would move on to the next node in the list. Including the rest of the code needed for the method, the method would look as shown in Fig. 5.5.

```
public void printRecursive(Node ref) {
    if(ref != null) {
        System.out.println(ref.getData());
        printRecursive(p.getNext());
    }
}
```

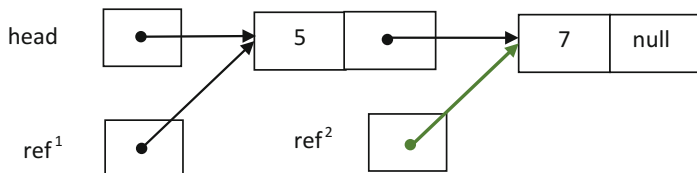
Fig. 5.5 `printRecursive` method

Although complete contours could be drawn to show each instance of the method as it is invoked, since the only variable (besides the return location) would be `ref`, it makes sense not to include the entire contour as discussed earlier in this chapter. Instead, only the variable `ref` will be drawn with a superscript to indicate which invocation is currently being used.

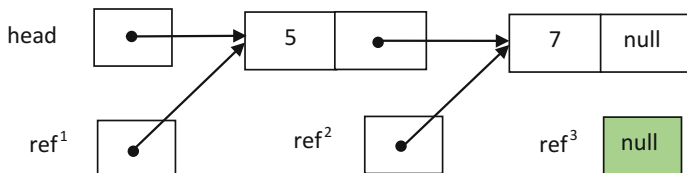
To invoke the above method, the invocation `printRecursive(head)` can be used. Upon entering the `printRecursive` method, `ref` will refer to the same node as `head`:



Notice that `ref` has a superscript of 1 indicating that it is in the first call to the `printRecursive` method. Since `ref` is not equal to `null`, the contents of the node are output and the method is invoked a second time. However, instead of `head` being sent as the argument, the contents of the next cell that `ref` is referring to is sent to the second invocation of `printRecursive`. The result is that `ref` in the second invocation will be referring to the node that contains the 7 as shown in the following diagram:



Notice that the second occurrence of `ref` has a superscript of 2 indicating that it is in the second call to the `printRecursive` method. Since `ref2` is not equal to `null`, the contents of the node are output and the method is invoked a third time. Again, the contents of the next cell that `ref2` is referring to are sent as an argument to the third invocation of `printRecursive`. The result is that `ref3` will contain a `null` as shown in the following diagram:



Since `ref`³ contains a `null`, which is the terminal case, no more recursion will take place. Instead, the third call to `printRecursive` will return to the second invocation, the second will return to the first, and the first will return back to the main program. Further, each of the variables for `ref` will be deallocated and the list will have been successfully output. Note that the implementation of recursion will be demonstrated in the complete program in the following section.

5.9 Complete Program: Outputting Items in the List Using Recursion

As in Chaps. 2–4, the Complete Program section will include generic classes. The concept of outputting the data in the list using recursion described in Sect. 4.8 will be implemented with a generic class so that the list of strings can be printed recursively.

First, the `Node` class shown in Fig. 5.1 will be rewritten so the `NodeGeneric` class can be used for any objects. As before, type `int` for the data in the node will be replaced by the type parameter `T`. Then, `NodeGeneric < T >` is going to be the reference type for the node instead of `Node`. The complete `NodeGeneric < T >` class is shown in Fig. 5.6.

```
public class NodeGeneric<T> {
    private T data;
    private NodeGeneric<T> next;

    public NodeGeneric() {
        this(null);
    }

    public NodeGeneric(T data) {
        this.data = data;
        next = null;
    }

    public T getData() {
        return data;
    }

    public void setData(T data) {
        this.data = data;
    }

    public NodeGeneric getNext() {
        return next;
    }

    public void setNext(NodeGeneric<T> next) {
        this.next = next;
    }
}
```

Fig. 5.6 `NodeGeneric < T >` class

Since strings stored in the list will be printed recursively, small changes will be made to the `printRecursive` routine described in Sect. 5.8. All that is needed to be done is to change the data type of the parameter `ref` from `Node` to `NodeGeneric < String >` shown below:

```
public static void printRecursive(NodeGeneric < String > ref) {  
    if(ref != null) {  
        System.out.println(ref.getData());  
        printRecursive(ref.getNext());  
    }  
}
```

Next, a main program which calls the `printRecursive` method is implemented. The test program shown in Sect. 5.7 is rewritten to accept strings from the user and print the items using the `printRecursive` routine instead of using a `for` loop. The following is the complete program including the `printRecursive` method located in the same class.

```
import java.util.*;  
  
class PrintRecStr {  
    public static void main(String[] args) {  
        Scanner scanner;  
        scanner = new Scanner(System.in);  
  
        String str;  
        NodeGeneric < String > head, newN;  
  
        head = null;  
        System.out.println();  
        System.out.print("Enter \"stop\" to stop: ");  
        str = new String(scanner.next());  
        while(!str.equals("stop")) {  
            newN = new NodeGeneric < String > (str);  
            newN.setNext(head);  
            head = newN;  
            System.out.print("Enter \"stop\" to stop: ");  
            str = scanner.next();  
        }  
        System.out.println();  
        printRecursive(head);  
    }  
}
```

```
public static void printRecursive(NodeGeneric < String > ref) {  
    if(ref != null) {  
        System.out.println(ref.getData());  
        printRecursive(ref.getNext());  
    }  
}  
}
```

When the main program is compiled and executed, it will prompt for various words. Whenever the user types the word "stop", it will output all the string values entered in reverse order.

```
Enter a word, "stop" to end: node  
Enter a word, "stop" to end: link  
Enter a word, "stop" to end: recursive  
Enter a word, "stop" to end: list  
Enter a word, "stop" to end: stop  
  
list  
recursive  
link  
node
```

So far in this chapter, the concepts of references and nodes have been introduced, and a simple linked list has been created and output both iteratively and recursively. However, there are far more complicated processes that can be performed with linked lists as will be explored in subsequent chapters. But before proceeding, it is important that the reader have a firm foundation in the fundamentals, so if there is any concern be sure to review this chapter again before proceeding.

5.10 Summary

- References can be used to implement linked lists.
- Linked lists are made up of objects that typically contain a data section and a reference section, which are often called `data` and `next`.
- If contours are not drawn to represent objects in a link list, one should note that the arrows drawn are not pointing to the individual data members, but rather to the object or node in its entirety.
- A reference that does not refer to anything is known as a `null` reference.
- Linked lists can be unordered or ordered.

- When manipulating references, it is usually better to create new references before breaking old references.
- A reference variable of the same type as `head`, such as `ref`, should be used to traverse the list.

5.11 Exercises (Items Marked with an * Have Solutions in Appendix B)

1. Suppose a linked list was created using the `Node` class described in Fig. 5.1 and contains three nodes with data 34, 26, and 78. Further, `head` references the first element.

*A. Draw a diagram of the linked list described above.

*B. Describe the effect of the following code segment and draw a diagram of the linked list.

```
Node curr;  
curr = head.getNext();  
curr.setData(9);
```

C. Starting from the original list above containing 34, 26, and 78, describe the effect of the following code segment and draw a diagram of the linked list.

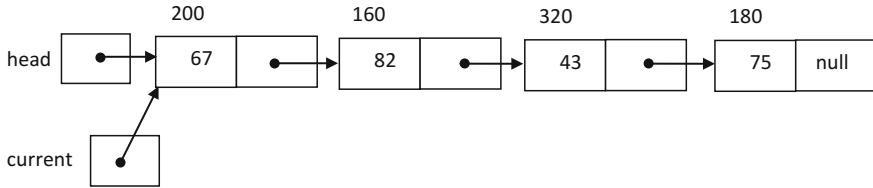
```
Node nodeRef;  
nodeRef = head.getNext();  
head.setNext(nodeRef.getNext());
```

D. Starting from the original list above containing 34, 26, and 78, describe the effect of the following code segment and draw a diagram of the linked list.

```
Node temp;  
temp = new Node(51);
```

```
temp.setNext(head);
head = temp;
```

Using the linked list created with the `Node` class given in Fig. 5.1 and shown below, answer the following questions, 2 through 4. The values above each node represent the address of the nodes and for questions that have an answer as an address, represent the address using parentheses, such as (300).



2. State if the syntax of the following statements is correct. If incorrect, explain why.

A. `current.getNext().getNext() = head;`

*B. `head.setNext(current.getData().getNext());`

C. `int ptr = 45;`
`head.setNext() = ptr;`

D. `head.getNext().getNext().getNext().getNext()`

3. Find the values of following expressions:

A. `current`

*B. `current.getNext().getNext()`

C. `current.getData()`

D. `head.getNext().getNext().getNext().getNext()`

4. Write Java statement(s) to do the following. For all the problems, start using the original list:
 - A. Make `current` reference the next node.
 - *B. Make `head` reference the node containing data 43.
 - C. Make `current` reference the last node.
 - D. Make `head` reference an empty list.
 - E. Change the value of the node containing 82 to 29.
 - F. Create and insert the node with data 10 after the node referenced by `current`.
 - G. Delete the node containing the number 82.
5. Write a code segment to remove the node containing the number 5 from the list shown in Fig. 5.4.
6. Write a code segment to add the node containing the number 3 between 9 and 5 in the list shown in Fig. 5.4.